

Command Injection (inyeccion de comandos de SO)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Ocurre cuando la app construye un comando del sistema operativo concatenando tu entrada (por ejemplo, para hacer ping a una IP que tu escribes) y lo ejecuta con `system()/exec()/popen()`. Como el shell no distingue 'el argumento que espera la app' de 'un comando nuevo', separadores como `;` `&&` `|` permiten añadir tu propio comando a continuación.

El programador confía en que el campo (host, IP, nombre de fichero) solo contendrá ese dato, sin validar ni escapar los caracteres especiales de shell. Misma causa raíz que `sqli` y `ssti`: mezclar dato de usuario con una instrucción que el sistema interpreta.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En funciones que 'suenan a' llamar herramientas del sistema: ping, traceroute, nslookup, convertir un fichero, hacer backup, comprobar disponibilidad de un host. Parámetros típicos: host, ip, domain, filename, target.

Cómo identificarla

- Un parámetro que hace de host/IP/dominio para una operación de red (ping, traceroute, whois).
- Un campo de nombre de fichero usado para convertir, comprimir o procesar con una herramienta externa.
- La respuesta tarda más de lo esperado al añadir `'&& sleep 5'` (señal de ejecución incluso sin salida visible).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade un comando inocuo tras un separador de shell: `127.0.0.1; id` (o `&& id`, `| id` según el shell) en el campo sospechoso.

Evidencia esperada: La salida de `id` (`uid=...` `gid=...`) aparece mezclada con la respuesta normal de la app. Eso confirma ejecución real, no solo un error.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS -X POST "$URL/api/ping" -H 'Content-Type: application/json' -d '{"host":"127.0.0.1; id"}'
```

Objetivo y contexto: Prueba de confirmación inocua: si 'host' llega a un ping del sistema, el ; encadena tu comando id sin romper el ping original.

Resultado esperado: La salida de id (uid=www-data...) mezclada en la respuesta confirma ejecución. El usuario que veas es con el que tendrás shell tras escalar.

Anatomía de los payloads

La primera prueba es `curl -sS -X POST "$URL/api/ping" -H 'Content-Type: application/json' -d '{"host":"127.0.0.1; id"}'`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

La salida de id (uid=... gid=...) aparece mezclada con la respuesta normal de la app. Eso confirma ejecución real, no solo un error.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica el campo que sospechas que llega a una llamada de sistema.
- Confirma con un comando inocuo primero (id/whoami/hostname); si no ves output, usa sleep/ping hacia tu máquina para confirmar a ciegas.
- Si hay una blacklist parcial, revisa filtros-incompletos y como normaliza la entrada antes de ejecutar.
- Si la salida ya es visible, prioriza una lectura acotada de configuración o una credencial reutilizable; solo pasa a reverse-vs-bind si una shell aporta algo.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app usa una llamada de sistema PARAMETRIZADA (pasa argumentos como lista, no como string concatenado) o una librería nativa en vez de invocar el binario del SO.

- El campo se valida contra un formato estricto (solo dígitos para un ID, regex de IP válida) antes de usarse.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Un parametro llega sin filtrar a una llamada al sistema operativo (ping, convert, backup); añades un separador de shell y ejecutas lo tuyo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(command-injection\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.